Data Distribution Shifts and Monitoring

Post-deployment challenges for ML systems A Lecture on Post-Deployment Challenges — Course: Advanced Machine Learning Systems; Audience: Upper-Level Undergraduate & Graduate Students

Duangdao Wichadakul

Lecture Agenda: Key Topics to Guide Today

Overview of parts covering causes, shifts, monitoring, and summary

1. Part 1: Causes of ML System Failures
2. Part 2: Data Distribution Shifts
3. Part 3: Monitoring and Observability
4. Part 4: Summary

Lecture Agenda: Key Topics to Guide Today

Overview of parts covering causes, shifts, monitoring, and summary

1. Part 1: Causes of ML System Failures
2. Part 2: Data Distribution Shifts
3. Part 3: Monitoring and Observability
4. Part 4: Summary

The **Fragility** of Production ML

Deployed models need continuous monitoring to protect business outcomes

A Deployed Model Is Not The End Of The Process; It's the Beginning

Deployment Starts Ongoing Responsibilities Like Monitoring And Maintenance

Models That Perform Well In Development Often Degrade In Production

Performance Drift And Environment Differences Reduce Effectiveness

Requires Continual Monitoring To Detect Issues That Can Cause Poor Business Outcomes

Detect Data Drift, Concept Drift, And Operational Failures Early

Defining an **ML System** Failure

When system expectations are violated

A failure occurs when one or more system expectations are violated.

Core definition of failure in ML systems.

Operational expectations relate to latency and throughput and are often easy to detect.

Performance metrics like latency and throughput are measurable.

ML performance expectations relate to prediction quality, and failures can be silent and harder to detect.

Prediction errors may not trigger obvious alerts.

Two Main Categories of **Failure**

Distinguish standard system issues from ML-specific risks

Time out

404 for Web

Segmentation fault

Out of memory error

etc.

??

ML Systems often fail silently

Software system failures:

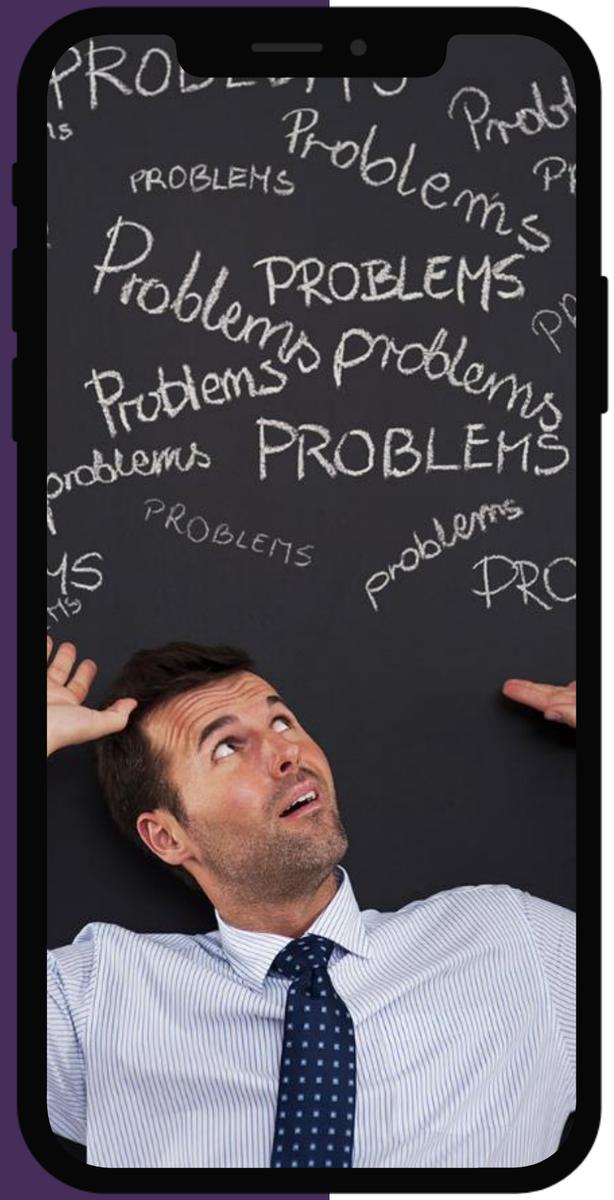
Traditional software and infrastructure issues addressed by standard engineering practices.

ML-specific failures:

Problems tied to data, model assumptions, or prediction-user interactions and often harder to detect.

Software System Failures

Operational, deployment, hardware, and dependency issues



Dependency failure: a dependent package or service breaks or is no longer maintained.

- A dependent package or service breaks or becomes unavailable, causing integration failures in the system.
- A dependent package or service is no longer maintained and accumulates security or compatibility issues over time, leading to failures.

Deployment failure: errors during deployment such as wrong model version or incorrect permissions.

- Errors during deployment such as pipeline failures, misconfigured CI/CD steps, or rollout issues that prevent proper release.
- Deploying the wrong model version to production can cause incorrect behavior and data mismatch across services.
- Incorrect permissions during deployment can block access to resources or cause services to crash due to denied operations.

Hardware failure: CPUs or GPUs malfunction, e.g., due to overheating.

- CPUs malfunction due to hardware faults, leading to compute errors or degraded performance over time.
- GPUs malfunction or fail under sustained load, causing training or inference jobs to abort unexpectedly.
- Overheating causes failures when cooling is insufficient, resulting in throttling, errors, or permanent damage.

Downtime or crashing: critical hosted components go down.

- Critical hosted components go down due to network issues, resource exhaustion, or cascading failures in dependent services.
- Services experience downtime or crashing when monitoring, autoscaling, or failover are misconfigured or insufficient.

Key insight: a Google study found undefined 60% of failures in a large ML pipeline were due to non-ML causes.

- A Google study found that approximately 60% of failures in a large ML pipeline were attributable to non-ML causes, such as infra and ops problems.
- Many failures in ML systems stem from operational, deployment, or dependency issues rather than model errors, highlighting the need for robust engineering.

ML-Specific Failures

Common causes of model breakdowns in production

1 **Production data differing from training data violates core ML assumptions.**

Model assumptions break when data shifts between train and deploy.

2 **Edge cases are rare extreme samples that can cause catastrophic mistakes.**

Unseen extremes lead to high-impact errors in rare situations.

3 **Degenerate feedback loops arise when model outputs influence future training data.**

Model behavior amplifies itself through retraining on biased outputs.

ML-Specific Failures

Common causes of model breakdowns in production

1 Production data differing from training data violates core ML assumptions.

Model assumptions break when data shifts between train and deploy.

ข้อมูลสอน (training data) มี distribution ต่างจากข้อมูลจริง (real-world data)

ตัวอย่างที่ดูเล็กน้อย เช่น emoji มีการทำ embedding ต่างกันระหว่างข้อมูลสอนกับข้อมูลจริง

ข้อมูลจริงมีการเปลี่ยนแปลงตลอดเวลา อาจจะ อย่างฉับพลัน อย่างช้าๆ หรือ ตามฤดูกาล เป็นต้น

Training-serving skew คือ??

หลายครั้ง data shifts อาจเกิดจาก internal errors เช่น bugs in data pipeline, การทำ imputation ที่ไม่ดีพอ การทำ feature extractions ที่ไม่สอดคล้องกันระหว่างการ train กับการ inference การใช้ model ผิดเวอร์ชัน หรือ bug ในแอปพลิเคชันเอง เป็นต้น

ML-Specific Failures

Common causes of model breakdowns in production

2 Edge cases are rare extreme samples that can cause catastrophic mistakes.

Unseen extremes lead to high-impact errors in rare situations.

Self-driving car ขับปลอดภัย 99.99% กล้านั่งไหม

Edge cases เป็นสิ่งสำคัญที่ทำให้ autonomous cars ยังไม่มีการใช้งานจริง

กรณีอื่นๆ เช่น medical diagnosis traffic control หรือแม้แต่ กลุ่มที่ไม่ได้เน้นความปลอดภัย เช่น chatbot พ่นคำหยาบคายออกมา เป็นต้น

ความแตกต่างระหว่าง Edge cases กับ Outliers?

Uber

This article is more than 7 years old

Self-driving Uber kills Arizona woman in first fatal crash involving pedestrian

Tempe police said car was in autonomous mode at the time of the crash and that the vehicle hit a woman who later died at a hospital

Sam Levin *and* Julia Carrie Wong *in San Francisco*

Mon 19 Mar 2018 22.48 GMT

Share

Prefer the Guardian on Google

Most viewed

Live
Andrew Mountbatten-Windsor arrested: King Charles says 'law must take its course' as ex-prince taken into custody - live

Andrew Mountbatten-Windsor arrested on suspicion of misconduct in public office

Why has former prince Andrew been arrested - and what happens now?

Live
Donald Trump repeats call for Iran to end nuclear program as world leaders

https://www.theguardian.com/technology/2018/mar/19/uber-self-driving-car-kills-woman-arizona-tempe

Distinguishing **Edge Cases** and Outliers

When unusual data matters for model performance

Outliers

Outliers are data points that differ significantly from other examples, such as a person jaywalking on a highway.

V

Edge Cases

Edge cases are examples where the model performs significantly worse than on other examples; **an outlier may or may not be an edge case.**

ML-Specific Failures

Common causes of model breakdowns in production

3 Degenerate feedback loops arise when model outputs influence future training data.

Model behavior amplifies itself through retraining on biased outputs.

- **A degenerate feedback loop occurs when model outputs are used as training data for future models**

  The model trains on its own predictions, reinforcing patterns.

- **For example, a recommender that ranks item A slightly higher gets more clicks on A**

  Higher rank leads to more user interactions for that item.

- **Retraining reinforces A's rank, causing exposure bias or popularity bias**

  The cycle amplifies certain items and limits diversity.

ML-Specific Failures

Common causes of model breakdowns in production

3 Degenerate feedback loops arise when model outputs influence future training data.

Model behavior amplifies itself through retraining on biased outputs.

Examples system with degenerate feedback loop: Recommendation systems, resume’ screening system with feature X.

Aliases of degenerate feedback loo[s: ”exposure bias”, “popularity bias”, “filter bubbles”, “echo chambers”

ML-Specific Failures

Common causes of model breakdowns in production

How to correct degenerate feedback loops arise when model outputs influence future training data.

Model behavior amplifies itself through retraining on biased outputs.

1. **Randomization**
   - A new video (in TikTok) will be randomized from an initial pool of traffic to be considered it is should be moved to a bigger pool.
   - Pro: improve diversity / Con: reduce user experience.

2. **Positional features**
   - Predict if a song will be clicked regardless of their rank in the list, if yes, this song's rank position feature will be set to be False.
   - Predict if users will be and consider the recommendation + Predict if users will click on that song.
   - etc.

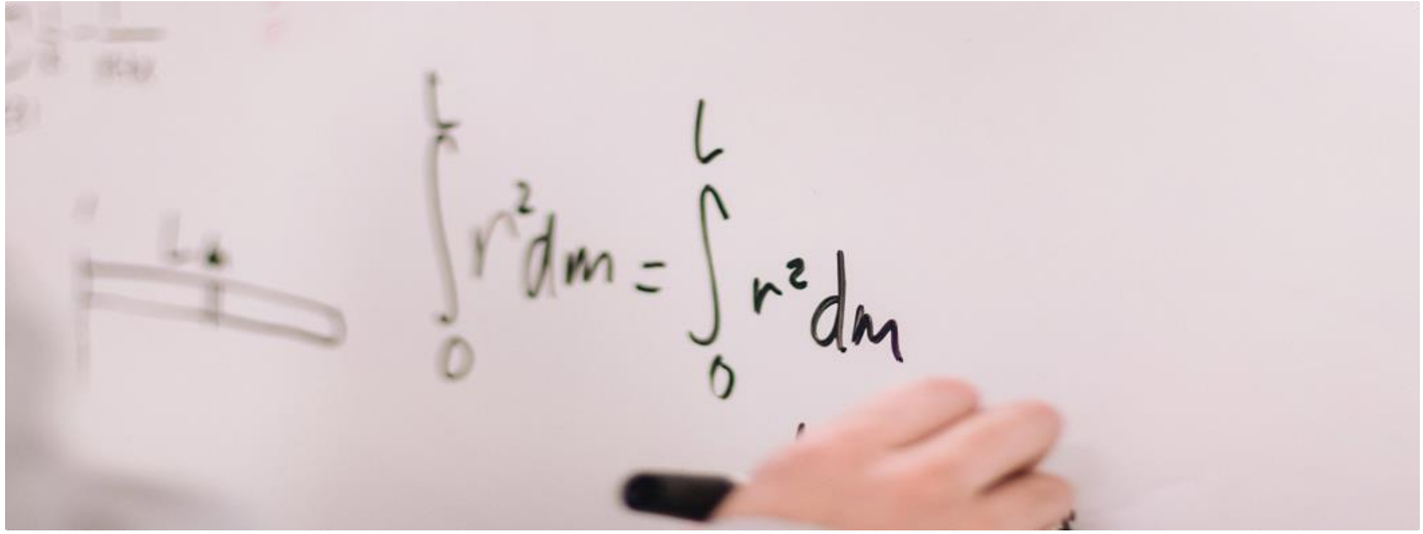
Lecture Agenda: Key Topics to Guide Today

Overview of parts covering causes, shifts, monitoring, and summary

1. Part 1: Causes of ML System Failures
2. Part 2: Data Distribution Shifts
3. Part 3: Monitoring and Observability
4. Part 4: Summary

Mathematical View of **Distribution Shift**

How changes in joint and conditional components alter data behavior



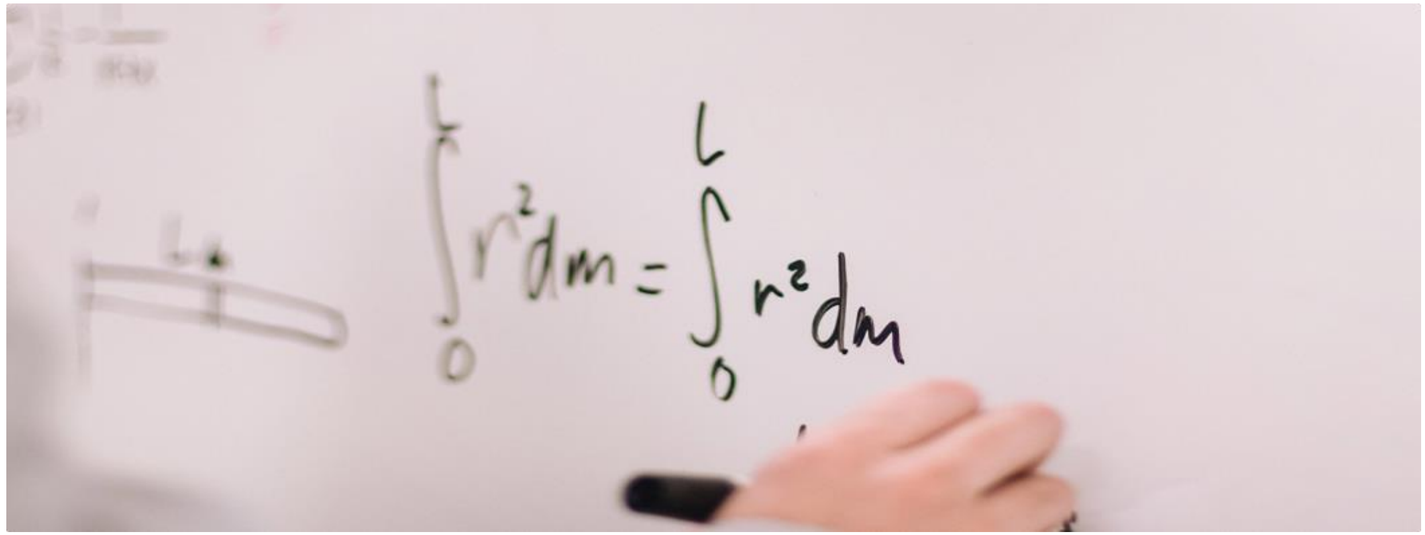
In supervised learning, a training data can be viewed as a set of samples from the joint distribution P(X, Y), which can be decomposed into two ways:

$P(X,Y) = P(Y|X) * P(X)$ and
$P(X,Y) = P(X|Y) * P(Y)$
where X are inputs and Y are labels.

Shifts occur when components of these decompositions change.

Mathematical View of **Distribution Shift**

How changes in joint and conditional components alter data behavior



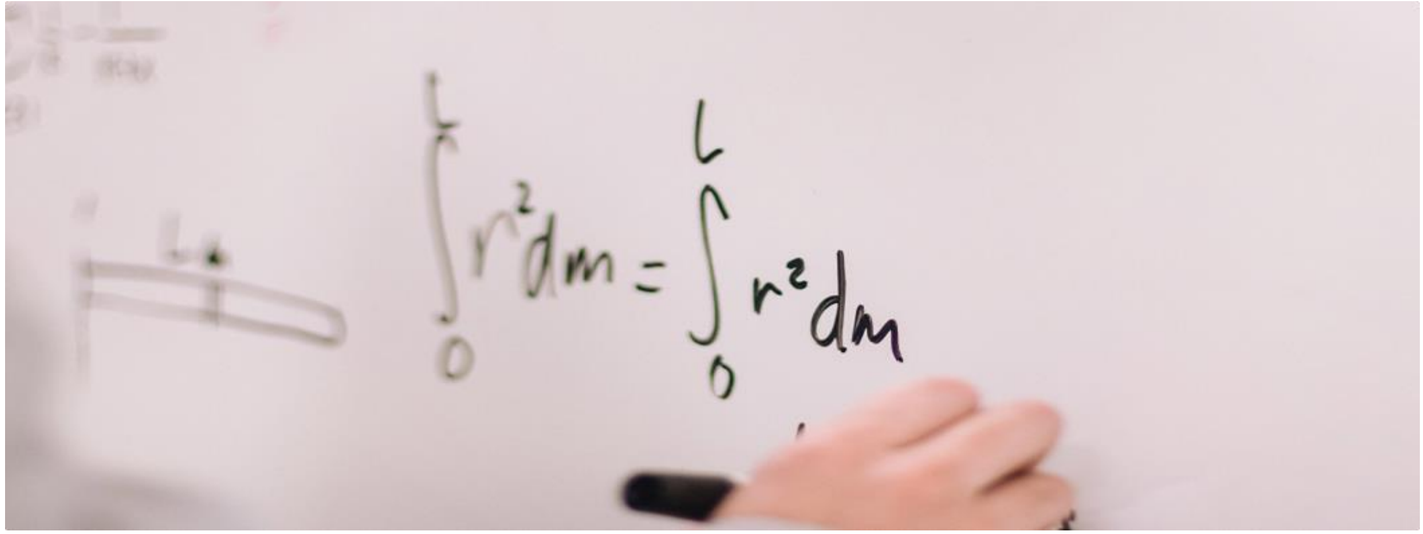
$P(X,Y) = P(Y|X) * P(X)$ and
$P(X,Y) = P(X|Y) * P(Y)$
where X are inputs and Y are labels.

P(X) denotes probability density of the input.
P(Y) denotes probability density of the output.
P(Y|X) denotes the conditional probability of an output given and input.

For example, the probability of an email is a spam, given the content of an email.

Mathematical View of **Distribution Shift**

How changes in joint and conditional components alter data behavior

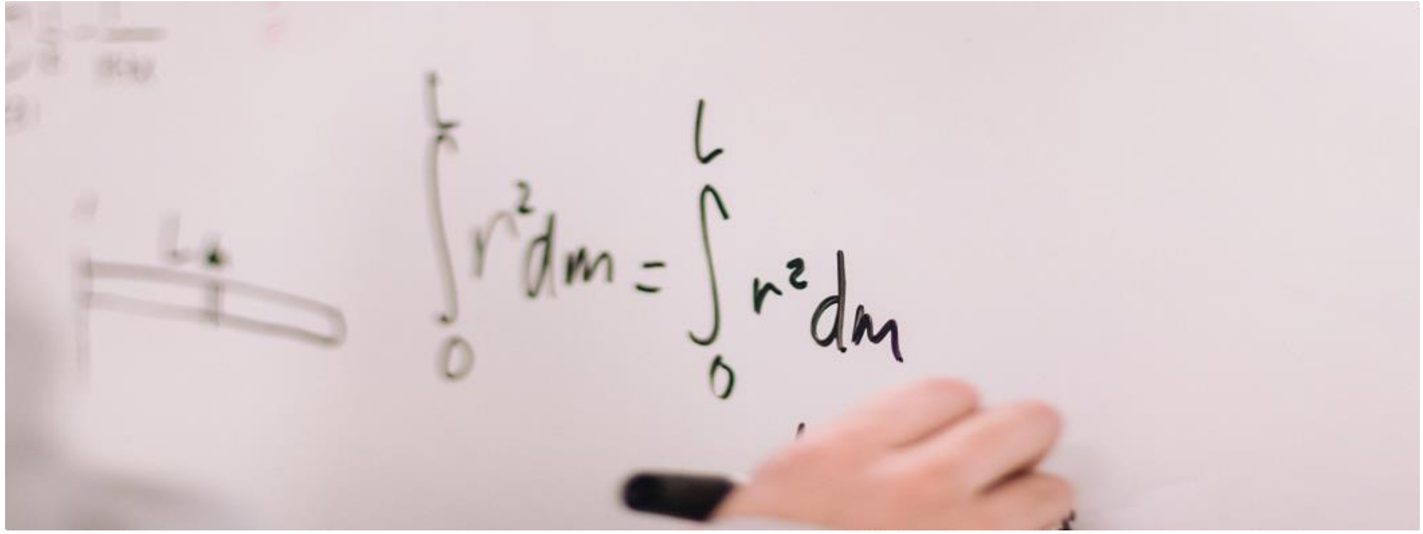


Three different shift and drift

1. Data drift (Covariate shift)
   When P(X) changes but P(Y|X) remains the same.
2. Label shift
   When P(Y) changes but P(X|Y) remains the same.
3. Concept drift
   When P(Y|X) changes but P(X) remains the same.

Mathematical View of **Distribution Shift**

How changes in joint and conditional components alter data behavior



Three different shift and drift

1. Data drfit (Covariate shift)
   The features (inputs) look different from the data the model trained on but the meaning of the data and the output goal have not changed.
2. Label shift
   Change in the frequency of different outcomes (labels), even if the input features and their relationships are the same.
3. Concept drift (most critical)
   The fundamental rules or patterns of the model learned are no longer valid in the real world. The same input might lead to different output.

https://medium.com/@byottech/explain-the-difference-between-concept-drift-data-drift-and-label-drift-f8a136a215c9

Covariate Shift

- Covariate shift is one of most widely studied forms of data distribution shift.
- In statistics, a covariate is an independent variable that can influence the outcome of a given statisticsl trial

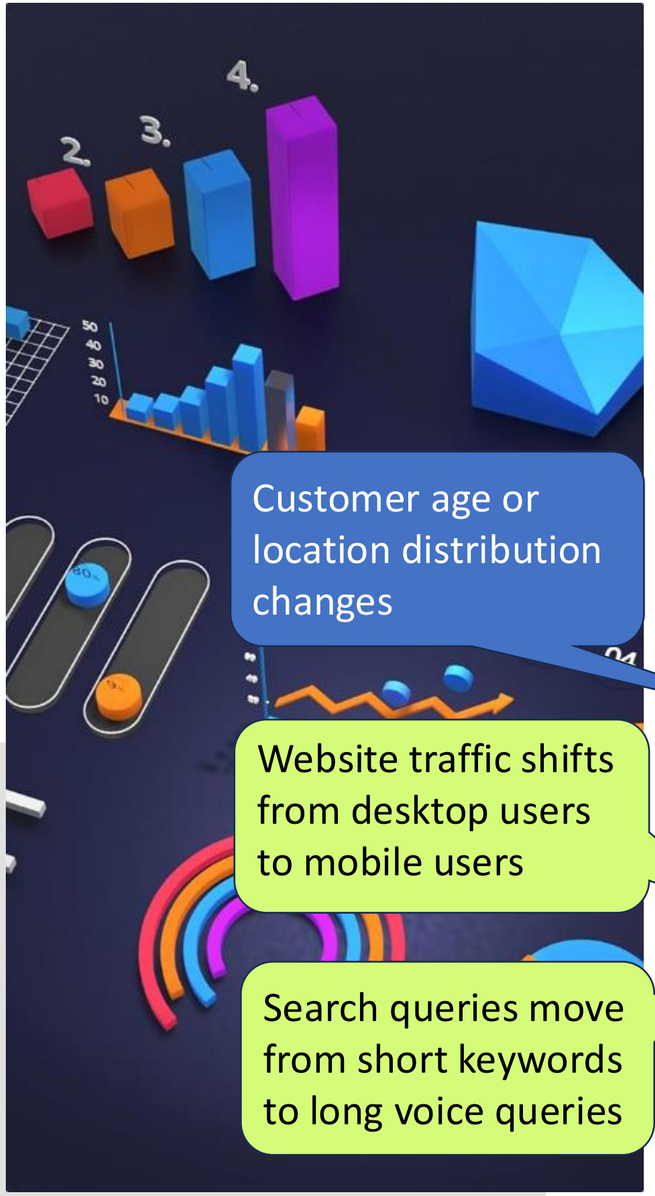
เราต้องการทดสอบว่าทำเลที่ตั้งมีผลต่อราคาคอนโดหรือไม่?

ราคาคอนโด เป็น direct interest

เราทราบว่าขนาดของคอนโดมีผลต่อราคา -> ขนาดของคอนโดเป็น covariate

ใน supervised learning, ราคาคอนโดเป็น direct interest ส่วนข้อมูลเข้า เช่น ขนาด ที่ตั้ง ที่จอดรถ เป็นต้น เป็น covariate variables

Covariate shift when inputs change but labels stay consistent.
Covariate shift occurs when P_source(X) ≠ P_target(X) but P_source(Y|X) = P_target(Y|X); the input distribution changes while the conditional mapping stays the same.



Real-World Examples of **Covariate Shift**

When input distributions change between development and production

Development: sample selection bias such as collecting data from a clinic specializing in older patients. Ex. Women age > 40 with breast cancer

Training inputs skewed toward older demographics

Development: Imbalanced Data From Oversampling Rare Classes To Aid Learning.

Artificial Class Distribution During Model Development.

Production: A Marketing Campaign Attracts A New Demographic, Changing Input Distribution While Conversion Probabilities Remain Constant.

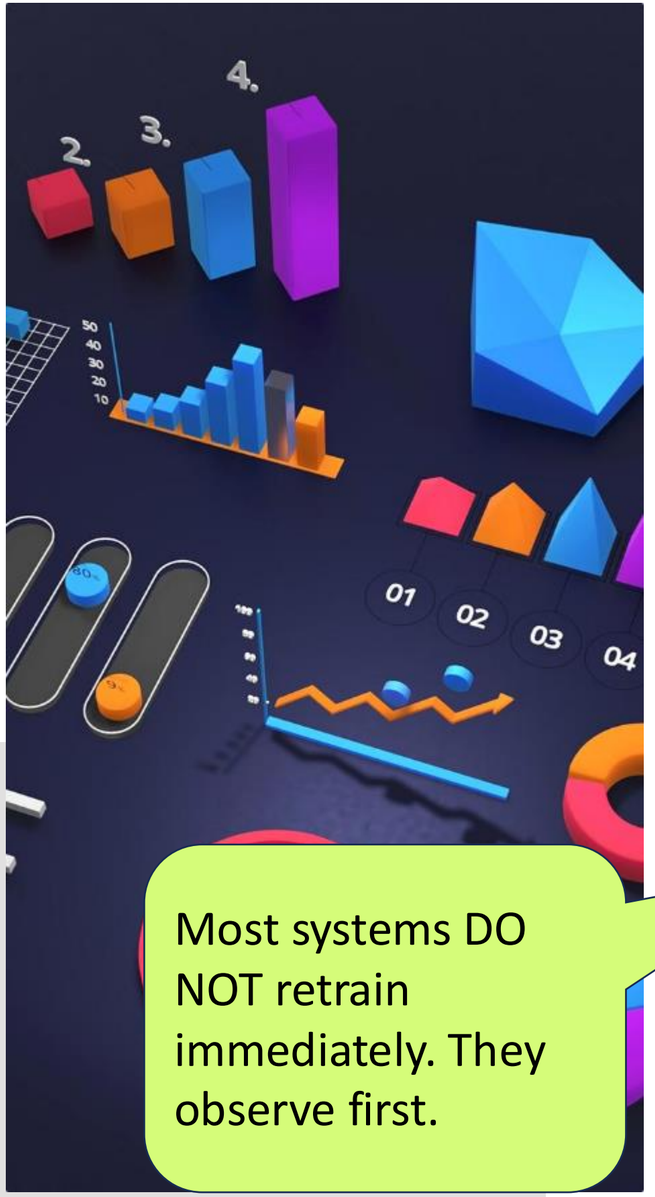
User Mix Shifts After Campaign Launch.

Covariate Shift Possible Handling

If we know in advance how the real-world input distribution will differ from the training input distribution, we can do the following

- **Importance weighting:** to train the model to work for the real-world data
  1) Estimate the density ratio between the real-world input distribution and the training input distribution
  2) Weight the training data according to this ratio for model training

ข้อจำกัด?



Most systems DO NOT retrain immediately. They observe first.

Covariate Shift Possible Handling

How AI systems detect data drift in real time

- Compare live input data with training data
- Monitor feature-level distribution shifts
- Use rolling time windows (hourly, daily)

Labels are **not required**, so detection is fast.

Common techniques include

- Population stability index (PSI)
- KL divergence or Jensen-Shannon distance
- Statistical tests on mean, variance, and percentiles

https://medium.com/@byottech/explain-the-difference-between-concept-drift-data-drift-and-label-drift-f8a136a215c9

Population Stability Index (PSI)

- PSI is a statistical tool used to measure how much the data distribution changes between two datasets.
- PSI is widely used in credit scoring, risk management, and financial modeling to ensure that the models remain valid and stable.

Production monitoring dashboards for model stability.

- Key terms in PSI
  - **Population**: A set of data points is being analyzed.
  - **Bins**: The dividing of population into bins. Ex. Low, Medium, High incomes
  - **Observed distribution**: The actual distribution of the data (for the time period we are analyzing)
  - **Expected distribution**: The reference distribution from the earlier time period

Population Stability Index (PSI) Formula

PSI is a univariate metric drift metric

$$PSI = \sum((\text{Observed} - \text{Expected}) * \ln \frac{Observed}{Expected}))$$

Where
Observed: proportion of data in each bin to the total observed for the current period
Expected: proportion of data in each bin to the total expected for the baseline period
ln: Natural logarithm

PSI < 0.1 : No significant change.
0.1 <= PSI < 0.25 : Moderate shft
PSI >= 0.25: Significant change

Higher PSI indicates the greater shift between distributions.

https://www.geeksforgeeks.org/data-science/population-stability-index-psi/

Population Stability Index (PSI) Workflow for a Dataset

Imagine you have a dataset with three features: age, income, and credit_score.
To monitor for covariate shift using PSI, you must calcuate PSI for each features separately.

1. Calculate PSI for the age feature

(1) Isolate: You take only the **age** column from your reference (training) dataset and your actual (live) dataset.
(2) Bin: You look at the **age** values from the reference data and create bins (e.g., 20-30, 30-40, 40-50, etc.).
(3) Calculate: You apply the PSI formula using the percentages of *age* values falling into these bins for both datasets.
(4) Result: You get one PSI value, let's say **PSI_age = 0.05** (stable).

Population Stability Index (PSI) Workflow for a Dataset

Imagine you have a dataset with three features: age, income, and credit_score. To monitor for covariate shift using PSI, you must calcuate PSI for each features separately.

2. Calculate PSI for the income feature

(1) Isolate: You take only the **income** column from both datasets.
(2) Bin: You look at the **income** values from the reference data and create bins (e.g., $30k-$50k, $50k-$70k, etc.).
(3) Calculate: You apply the PSI formula using the percentages of *income* values falling into these bins for both datasets.
(4) Result: You get a second, independent PSI value, let's say **PSI_income = 0.28** (major shift).

Population Stability Index (PSI) Workflow for a Dataset

Imagine you have a dataset with three features: age, income, and credit_score. To monitor for covariate shift using PSI, you must calcuate PSI for each features separately.

3. Calculate PSI for the credit_score feature

(1) Isolate: You take only the **credit_score** column from both datasets.
(2) Bin: You create bins for the credit scores based on the reference data.
(3) Calculate: You apply the PSI formula using the percentages of *credit_score* values falling into these bins for both datasets.
(4) Result: You get a third PSI value, let's say **PSI_score = 0.12** (minor shift).

From the three features, the income values need to be checked

Label Shift

also known as prior probability shift, prior probability shift, or target shift

When target label proportions change but conditional features stay the same

Definition: Label shift occurs when P_source(Y) ≠ P_target(Y) but P_source(X|Y) = P_target(X|Y)

1

Preserves conditional feature distributions while label priors change.

Example: a preventive drug reduces disease prevalence while the feature distribution conditioned on disease remains the same

2

Illustrates change in label prevalence without altering X given Y.

Real-World Examples of **Label Shift**

When input distributions change between development and production

1. Fraud rate suddenly increases during festive seasons

r

2. Conversion rate spikes due to discounts to campaigns

3. Spam volume increases after a major event

https://medium.com/@byottech/explain-the-difference-between-concept-drift-data-drift-and-label-drift-f8a136a215c9

Label Shift Possible Handling

How AI systems detect label drift in real time

- Monitor class frequency changes
- Track base rate shifts in labels
- Compare predicted class distribution with actual labels

Typical fixes include

- Threshold adjustment
- Probability recalibration
- Cost-sensitive tuning

Detection requires label, but often less data than concept drift

https://medium.com/@byottech/explain-the-difference-between-concept-drift-data-drift-and-label-drift-f8a136a215c9

Label Shift Possible Handling

How AI systems detect label drift in real time

- Monitor class frequency changes
- Track base rate shifts in labels
- Compare predicted class distribution with actual labels

Typical fixes include

- Threshold adjustment
- Probability recalibration
- Cost-sensitive tuning

Adjust threshold for output probability

https://medium.com/@byottech/explain-the-difference-between-concept-drift-data-drift-and-label-drift-f8a136a215c9

Label Shift Possible Handling

How AI systems detect label drift in real time

- Monitor class frequency changes
- Track base rate shifts in labels
- Compare predicted class distribution with actual labels

Typical fixes include

- Threshold adjustment
- Probability recalibration
- Cost-sensitive tuning

Making the model's probability score reliable and trustworthy before using it.

https://medium.com/@byottech/explain-the-difference-between-concept-drift-data-drift-and-label-drift-f8a136a215c9

Label Shift Possible Handling

How AI systems detect label drift in real time

- Monitor class frequency changes
- Track base rate shifts in labels
- Compare predicted class distribution with actual labels

Typical fixes include

- Threshold adjustment
- Probability recalibration
- Cost-sensitive tuning

Using mathematical formula based on real=world costs to tell exactly the probability we should use.

https://medium.com/@byottech/explain-the-difference-between-concept-drift-data-drift-and-label-drift-f8a136a215c9

Cost-sensitive tuning

Adjust your model decision threshold to account for both the new class proportions (from label shift). It is a post-training adjustment.

The process:

1. **Define misclassification costs:** You may define the cost of each type of error.
   - C(FN): Cost of False Negative (e.g, cost of misfraud = $500)
   - C(FP): Cost of False Positive (e.g., cost of incorrectly blocked transaction = $5)
2. **Estimate new label proportion:** For example, in the original trained model the fraud is 1% but now it becomes 10%, so, P_new(Y=1) = 0.1 instead of 0.01
3. **Tune the decision threshold:** by using this formula

   T = 1 / (1 + (CostRatio * OddsRatio))

   CostRatio = C(FN) / C(FP)

   OddsRatio = P_new(Y=1) / P_new(Y=0)

Cost-sensitive tuning

Example:

C(FP) = $5
C(FN) = $500
P_new(Y=1) = 0.1 (10% fraud)
P_new(Y=0) = 0.9

CostRatio = 500/5 = 100
OddsRatio = 0.1/0.9 = 0.1111..

From T = 1 / (1 + (CostRatio * OddsRatio))
Then T = 1 / (1 + (100 * 0.1111))
= 0.0825

]

> Decision rule → if score > 0.0825 then predicts "Fraud"

Realted examples of post-tuning for cost-sensitive learning https://scikit-learn.org/stable/auto_examples/model_selection/plot_cost_sensitive_learning.html

Sudden drift: housing prices for the same features change drastically due to an economic event.

Abrupt distribution change after an economic shock.

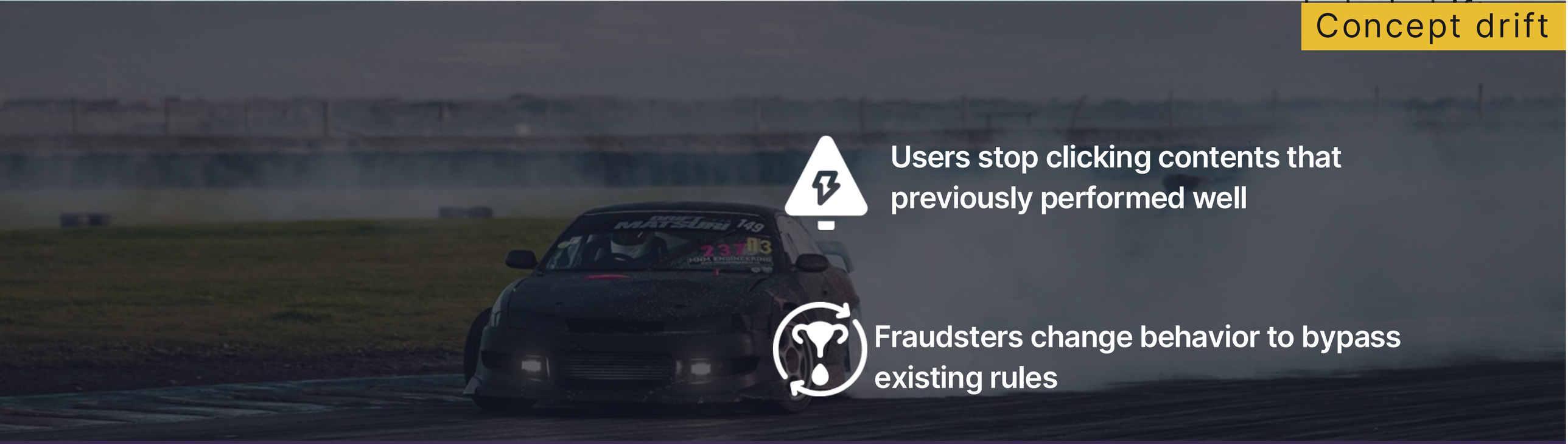
Real-World Examples of **Concept Drift**

When models face changing data patterns

When P(Y|X) changes but P(X) remains the same.

Seasonal or cyclic drift: rideshare pricing or retail purchasing habits change with time and may require different models.

Predictable periodic shifts in behavior or demand.



Users stop clicking contents that previously performed well

Fraudsters change behavior to bypass existing rules

Customer buying decisions change due to economic shifts

Real-World Examples of **Concept Drift**

When models face changing data patterns

When P(Y|X) changes but P(X) remains the same.

The same inputs lead to different outcome.

Concept Shift Possible Handling

How AI systems detect data drift in real time

- Monitor model performance metrics over time
- Track drops in accuracy, F1, AUC, and precision
- Observe increasing prediction errors or miscalibration

Advanced drift detectors

- Page-Hinkley test
- DDM and EDDM
- ADWIN (Adaptive Windowing)

Concept drift detection usually requires labels, so, it may be delayed.

When confirmed, model retraining is necessary.

https://medium.com/@byottech/explain-the-difference-between-concept-drift-data-drift-and-label-drift-f8a136a215c9

Method	Used Most For...	Primary Strength	Main Weakness
ADWIN	**General-purpose, robust default choice.**	Automatically adapts window size.	More complex and computationally intensive.
DDM / EDDM	**Simple, fast baseline detection.**	Lightweight, fast, and intuitive.	Relies on fixed-window assumptions; less robust.
Page-Hinkley Test	**Detecting abrupt changes in a signal's mean.**	Statistically sound for change-point detection.	Less flexible for complex or gradual drift.

Concept Shift Advanced Drift Detectors

ADWIN (Adaptive) Windowing

Primary Strength: The "Adaptive" Window

- If you choose a **small window** , you can react quickly to sudden, abrupt drift, but you might get a lot of false alarms from random noise.
- If you choose a **large window** , you will be more robust to noise, but you will be very slow to detect gradual drift.

Adaptive windowing is the killer feature of ADWIN compared to DDM and Page-Hinkley

Concept Shift Advanced Drift Detectors

ADWIN (Adaptive) Windowing

ADWIN Adaptive Windowing does not have a fixed window size. It automatically grows its window when the data is stable and shrinks it dramatically when a change is detected. This means

- It is effective against **both fast and gradual drift.**
- It requires **less parameter tuning** and guesswork from the user.
- It automatically **forgets old data** after a drift, which is crucial for adapting to the new concept.

Concept Shift Advanced Drift Detectors

ADWIN (Adaptive) Windowing

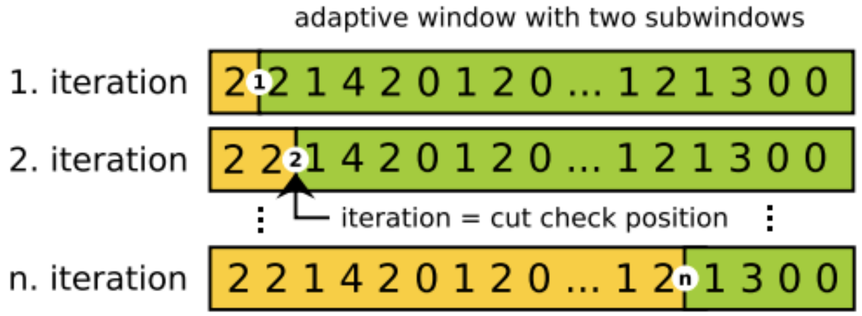


Figure 1: Iterations of the cut check procedure in ADWIN

Add How ADWIN WORK inside

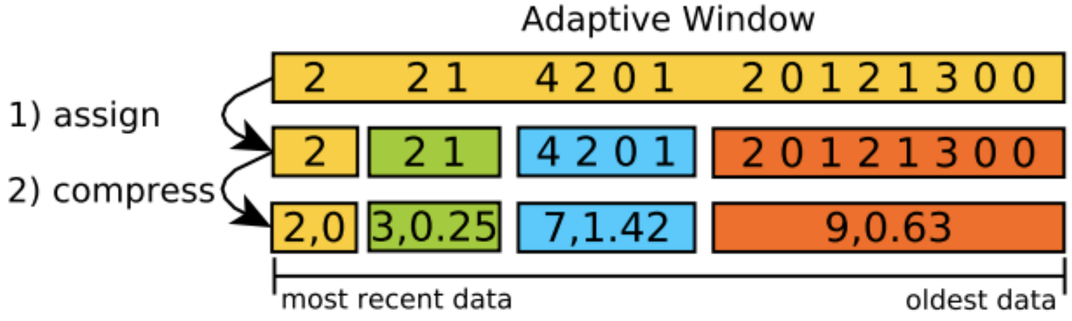


Figure 2: Exponential histogram: We first assign tuples to buckets of exponential size. We then compress buckets and store sums and variances only.

Traub2018scotty (2017) https://github.com/TU-Berlin-DIMA/parallel-ADWIN

Concept Shift Advanced Drift Detectors

DDM and EDDM : The Simple, Fast Baseline

Primary Strength: Simplicity and Speed

- **Easy to implement:** The logic is straightforward and computationally very cheap.
- **Intuitive:** Monitor the model's error rate and its standard deviation
- **Good for abrupt drift:** DDM is good at detecting a sudden spike in errors

Limitations: They are less effective with gradual shift (though EDDM tries to address this) and can be more prone to false alarm compared to ADWIN.

Conclusion: DDM is a great "first-step" detector. If you need something extremely lightweight for a resource-constrained environment (like IoT) or want a simple benchmark to compare against, DDM/EDDM are excellent choices.

Time Scale Window for detecting Shifts

Balance speed and stability when choosing window size

Shift happen at different rates

- **Shifts can also happen across two dimensions:**
  -> Spatial
  -> Time

Time Scale Window for detecting Shifts

Balance speed and stability when choosing window size

1

Short Time Windows Enable Faster Detection But Increase False Alarms Due To Normal Variance.

Faster Alerts Can Catch Quick Shifts But Risk Noise-induced False Positives.

2

Long Time Windows Are More Robust To Fluctuations But Slower To Detect Gradual Shifts; Choose Window Size Carefully To Capture Relevant Cycles Such As Weekly Patterns.

Longer Windows Reduce Noise Sensitivity But May Delay Response To Trends.

Time Scale Window for detecting Shifts

Balance speed and stability when choosing window size

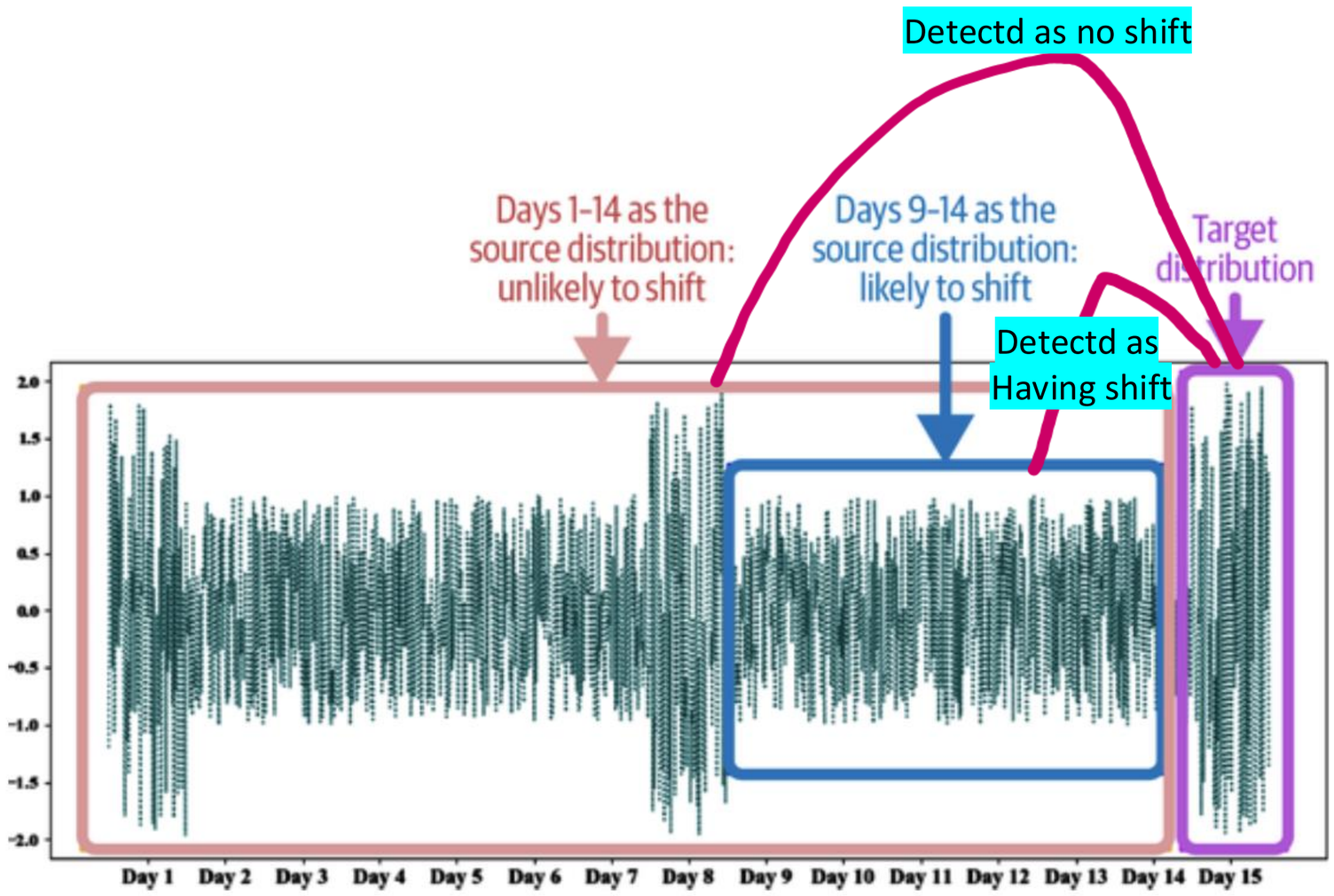


Figure 8-3. Whether a distribution has drifted over time depends on the time scale window specified

Source: Chip Huyen, Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications. Chapter 8. O'Reilly Media, Inc. 2022

Time Scale Window for detecting Shifts

Balance speed and stability when choosing window size

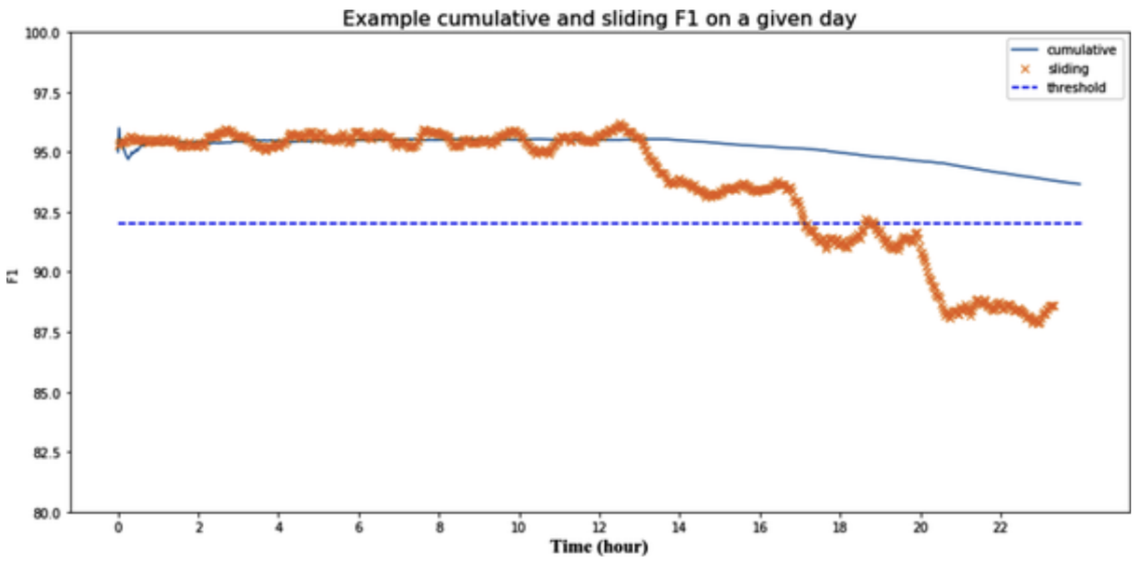


Figure 8-4. Cumulative accuracy hides the sudden dip in accuracy between hours 16 and 18. Source: Adapted from an image by MadeWithML

How to deal with the seasonality of the market?

Source: Chip Huyen, Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications. Chapter 8. O'Reilly Media, Inc. 2022

https://eng.lyft.com/how-to-deal-with-the-seasonality-of-a-market-584cc94d6b75

General Data Distribution Shifts

When features or labels change, update pipelines and retrain models

Feature change: adding or removing features or changing value ranges may require pipeline updates.

May need data preprocessing and validation updates.

1

Ex: Age was year now it becomes month

Label schema change: adding new classes or splitting existing classes often requires architecture changes and retraining.

Can require model redesign and full retraining.

2

label schema change, both P(Y) and P(X|Y) change

Ex. Regression tasks, the changed possible label value range.

Ex. classification tasks, a new class is introduced or some existing classes were are removed.

Kolmogorov-Smirnov (K-S) or KS Test

Nonparametric two-sample comparison for one-dimensional data

Does not require any parameters of the underlying distribution to work

1. **The K-S test is a nonparametric two-sample test comparing cumulative distribution functions**

   Compares full CDFs without distributional assumptions.

2. **it requires one-dimensional data (scalar numbers)**

   Best for 1D scalar measurements.

3. **Useful to detect label or prediction shift**

   Cons: it won't work for high-dimensional data

Other two-sample tests

It is highly recommended that you reduce the dimensionality of your data before performing a two-sample test on it.

1. **Least-Squares Density Difference**

   Based on least-squares density-difference estimation method

2. **Maximum Mean Discrepancy (MMD) (Gretton et al. 2020)**

   A kernel-based technique for multi-variate two-sample testing. Popular in research.

3. **Learned Kernel MMD (Liu et al. 2020)**

   A variant of MMD

etc.

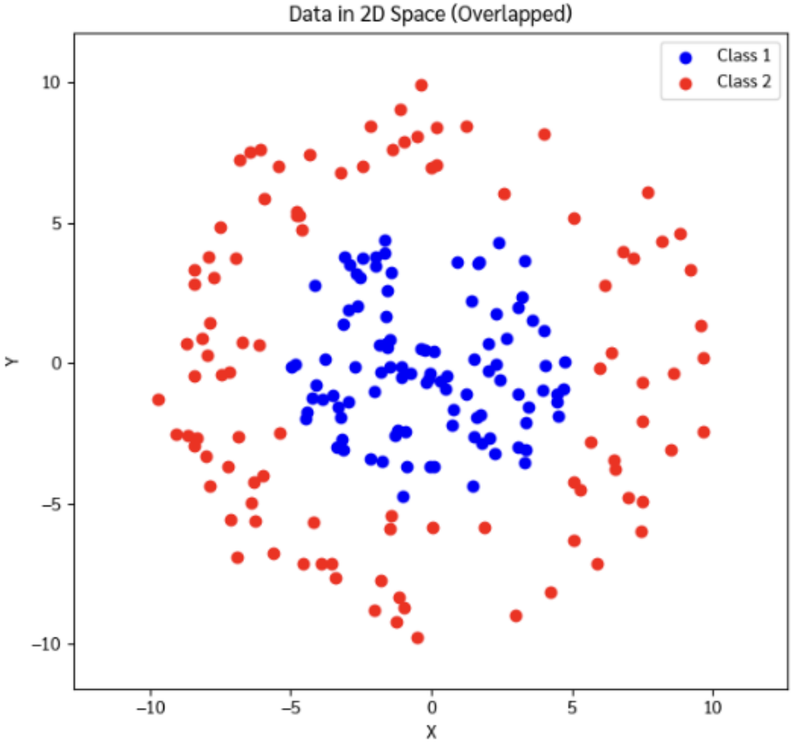
MMD

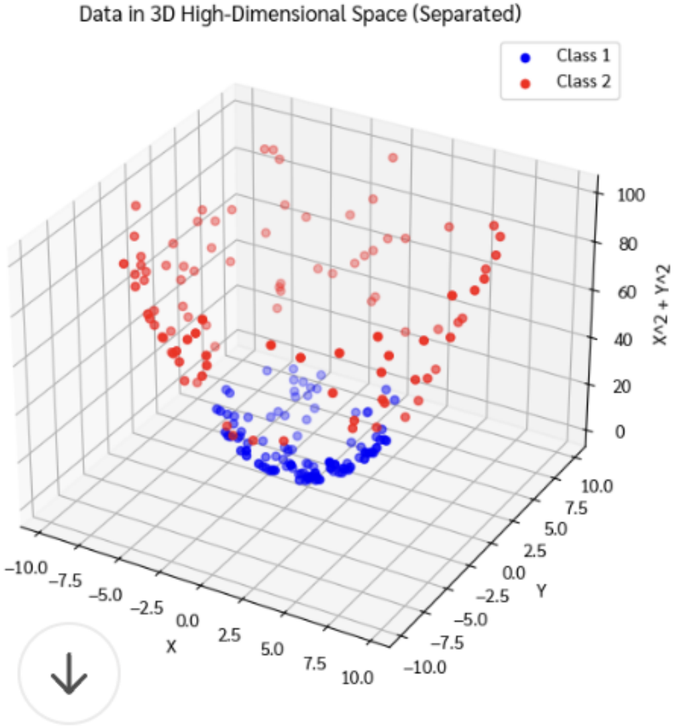
How MMD Works, Step-by-Step

Step 1: The "Kernel Trick" - Mapping to a Higher Dimension

A kernel function takes two data points from the original space (e.g., two 512-dimensional image embeddings) and calculates their "similarity." A common choice is the Gaussian (or RBF) kernel, which says "points that are close together in the original space are very similar, and points that are far apart are not."

While it feels like a simple similarity calculation, the kernel is implicitly mapping every data point into an infinitely-high-dimensional feature space.





How MMD Works, Step-by-Step

Step 2: Calculating the "Mean Embedding"

This "center of gravity" in the high-dimensional feature space is the **Mean Embedding** . It is the rich "fingerprint" capturing not just the average of the distribution but also its shape, bumps, and other complex properties.

We now have two mean embeddings:

Mean Embedding of P: The center of gravity for the first dataset (e.g., your original images).

Mean Embedding of Q: The center of gravity for the second dataset (e.g., your new, potentially drifted images).

Step 3: Measuring the Discrepancy

$$\mathbf{MMD^2(P,Q) = \| Mean_Embedding(P) - Mean_Embedding(Q) \|^2}$$

If MMD close to 0, then have no drift between P and Q

MMD

Alibi-detect uses MMD to detect drift by determining if this number is "large enough" to be significant. It does this using **hypothesis testing**.

Null Hypothesis (H_0): The two datasets (P and Q) come from the same distribution.

Permutation Testing:

It pools all the data from P and Q together.
It shuffles this combined dataset and randomly splits it into a new fake-P and fake-Q.
It calculates the MMD between this fake-P and fake-Q.
It repeats this shuffling and calculating hundreds of times. This gives a distribution of MMD values

The Decision: The detector compares actual MMD value (from the original, unshuffled data) to this distribution of random MMD values.

If your actual MMD value is typical and falls within the range of the random values, the **p-value will be high** . We cannot reject the null hypothesis, so we conclude there is **no drift** .
If your actual MMD value is an extreme outlier and is much larger than almost all of the random values, the **p-value will be very low** (e.g., < 0.05). This means it's extremely unlikely you'd get a discrepancy this large by chance. We reject the null hypothesis and conclude there **is drift** .

ALIBI's implemented outlier detection algorithms

ALIBI DETECT

CI no status | docs passing | codecov 69% | python 3.9 | 3.10 | 3.11 | 3.12
pypi v0.13.0 | conda-forge v0.11.4 | license not identifiable by github | chat on slack

a source-available Python library focused on **outlier**, **adversarial** and **drift** detection.

https://github.com/SeldonIO/alibi-detect

Outlier Detection

Detector	Tabular	Image	Time Series	Text	Categorical Features	Online	Feature Level
Isolation Forest	✓				✓		
Mahalanobis Distance	✓				✓	✓	
AE	✓	✓					✓
VAE	✓	✓					✓
AEGMM	✓	✓					
VAEGMM	✓	✓					
Likelihood Ratios	✓	✓	✓		✓		✓
Prophet			✓				
Spectral Residual			✓			✓	✓
Seq2Seq			✓				✓

ALIBI DETECT

CI no status | docs passing | codecov 69% | python 3.9 | 3.10 | 3.11 | 3.12
pypi v0.13.0 | conda-forge v0.11.4 | license not identifiable by github | chat on slack

a source-available Python library focused on **outlier**, **adversarial** and **drift** detection.

https://github.com/SeldonIO/alibi-detect

ALIBI's implemented drift detection algorithms

Drift Detection

Detector	Tabular	Image	Time Series	Text	Categorical Features	Online	Feature Level
Kolmogorov-Smirnov	✓	✓		✓	✓		✓
Cramér-von Mises	✓	✓				✓	✓
Fisher's Exact Test	✓				✓	✓	✓
Maximum Mean Discrepancy (MMD)	✓	✓		✓	✓	✓	
Learned Kernel MMD	✓	✓		✓	✓		
Context-aware MMD	✓	✓	✓	✓	✓		
Least-Squares Density Difference	✓	✓		✓	✓	✓	
Chi-Squared	✓				✓		✓
Mixed-type tabular data	✓				✓		✓
Classifier	✓	✓	✓	✓	✓		
Spot-the-diff	✓	✓	✓	✓	✓		✓
Classifier Uncertainty	✓	✓	✓	✓	✓		
Regressor Uncertainty	✓	✓	✓	✓	✓		

ALIBI's implemented adversarial detection algorithms

ALIBI DETECT

CI no status | docs passing | codecov 69% | python 3.9 | 3.10 | 3.11 | 3.12
pypi v0.13.0 | conda-forge v0.11.4 | license not identifiable by github | chat on slack

a source-available Python library focused on **outlier**, **adversarial** and **drift** detection.

https://github.com/SeldonIO/alibi-detect

Adversarial Detection

Detector	Tabular	Image	Time Series	Text	Categorical Features	Online	Feature Level
Adversarial AE	✓	✓					
Model distillation	✓	✓	✓	✓	✓		

Addressing Data Distribution Shifts

Retaining from scrtch with both old and new data/ Continuing training the existing model on new data (fine-tuing)

Retrain from scratch (stateless retraining) / Retraining from the last checkpoint (stateful retraining)

Retrain with what data, last 24 hour, last week, or from the point of detected drift?

Three main approaches to make a model work with a new distribution in production

เจอ drift แล้วทำอะไรต่อ

1. **Train model using massive datasets**

   This currently dominates research. Hope that dataset is large enough to for the model to learn a comprehensive distribution

2. **Adapt a trained model to a target distribution w/o requiring new labels**

   Mainly in research not yet adopted by the industry

3. **Retain the model using the labeled data from the target distribution**

   Usually done in the industry today

Addressing Data Distribution Shifts

Addressing data distribution shifts doesn't have to start after the shifts have happened. Ex: a feature rank has changed very quickly. We can also use the binning/ranging of ranks instead. (pro/con)

Design a system to make it easier for it to adapt to shifts. Ex: Develop models for specific cities with different characters.

Many ML failures today are still caused by human errors. Must try to identify these causes of errors first.

Lecture Agenda: Key Topics to Guide Today

Overview of parts covering causes, shifts, monitoring, and summary

1. Part 1: Causes of ML System Failures
2. Part 2: Data Distribution Shifts
3. Part 3: Monitoring and Observability
4. Part 4: Summary

Monitoring vs **Observability**: Know When and Understand Why

Contrast passive detection with active investigative visibility

Know When?

Monitoring

- Monitoring tracks metrics, dashboards, and alerts to determine when something goes wrong and is inherently passive.

Understand Why?

Observability

- Observability provides visibility to investigate what went wrong and enables asking new questions about system behavior without shipping new code.

Monitoring is all about metrics

ML systems are software systems

The first class of metrics, we must monitor are the operational metrics, monitoring the system health. They are generally divided into three levels

1. The network level the system is run on
2. The machine the system is run on
3. The application that the system runs

Examples of metrics:
Latency; Throughput; the number of prediction requests a model received in the last minute, hour, day; the percentage of requests returned with 2xx code; CPU/GPU utilization; memory utilization; etc.

Availability measured by “uptime”: the conditions determined whether the system is up defined in the service level objectives (SLOs) or service level agreements (SLAs).Ex. SLA may specify if the service is considered ”up” if it has a median latency less than 200ms and 99th percentile under 2 seconds.

ML-Specific Metrics

Track inputs, features, predictions, and accuracy to detect issues early

Harder to monitor

Easier to monitor

1

Raw Inputs: Original Unstructured Data Fed Into The System Are Hardest But Closest To Source To Monitor.

Hardest To Monitor But Most Faithful To The Source

2

Features: Structured Inputs After Processing; Easier To Monitor But Can Reflect Pipeline Bugs.

Can Reveal Preprocessing Or Pipeline Issues

3

Predictions: Model Outputs That Are Low-dimensional And Easy To Monitor As Proxies For Input Shifts.

Simple To Track And Indicative Of Upstream Drift

4

Accuracy: The Ground Truth Of Model Performance When Labels Are Available But Often High-latency.

Definitive But Frequently Delayed Metric

Monitoring Accuracy-Related Metrics

Track inputs, features, predictions, and accuracy to detect issues early

เรามอนิเตอร์อะไรได้บ้าง

Log and track all user feedbacks

Log any types of model prediction feedbacks: click, hide, purchase, upvote, downvote, favorite, bookmark, share, etc., even though the feedbacks can not be used to infer natural label directly, they can be used to indirectly infer the model performance. Why?

(1) มาตรวัดเกี่ยวกับประสิทธิภาพของแบบจำลอง

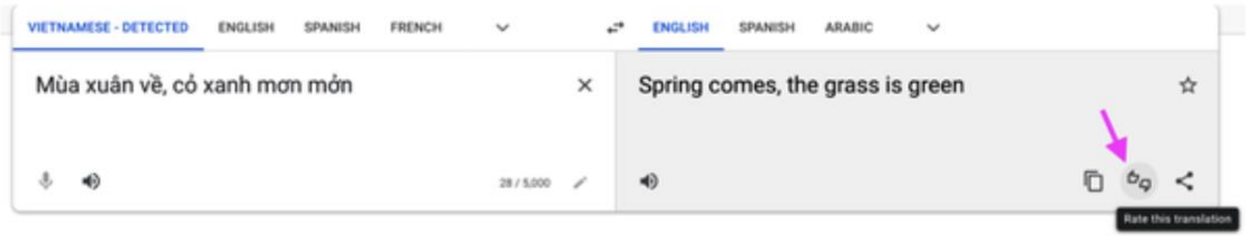


Figure 8-6. Google Translate allows users to upvote or downvote a translation. These votes will be used to evaluate their translation model's quality as well as to guide the labeling process.

Source: Chip Huyen, Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications. Chapter 8. O'Reilly Media, Inc. 2022

Monitoring Predictions

Track inputs, features, predictions, and accuracy to detect issues early

ตรวจสอบผลที่ทำนายเพื่อย้อนกลับไปอนุมานเหตุ (ข้อมูลเข้า)

Monitor predictions for distribution shifts

It is easier to compute two-sample tests to detect prediction distribution has shifted, if yes, we can infer input distribution shifts, assuming the mapping the input to output; the weights and biases, haven't changed.

Monitor predictions for any odds happening

Changes in accuracy-metrics might not become obvious for days or weeks, where a model predicting all False for 10 minutes (the odd results) can be detected fast.

Source: Chip Huyen, Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications. Chapter 8. O'Reilly Media, Inc. 2022

Monitoring Features

Track inputs, features, predictions, and accuracy to detect issues early

ตรวจสอบฟีเจอร์ทั้งหลายที่ใช้ ว่ามีอะไรเพี้ยนไปไหม

ML monitoring solutions in the industry focus on tracking changes in features

It tracks both features the model uses as inputs and the intermediate transformations from raw inputs into final features. Features are well structured, defined comparing with the raw inputs. What to do for feature monitoring:

- Feature validation (a.k.a. table testing or table validation or unit test for data) : ensuring that the features follow an expected schema. Ex:
  - If min, max, median values are in a normal range
  - If the values satisfy the regular expression format
  - If all the values of a feature belong to a pre-defined set

Source: Chip Huyen, Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications. Chapter 8. O'Reilly Media, Inc. 2022

Most Common Open Source Libraries for Basic Feature Validation

Great Expectations

python 3.10 | 3.11 | 3.12 | 3.13 pypi v1.11.0 downloads rate limited by upstream service build passing
pre-commit.ci passed codecov 84% DOI 10.5281/zenodo.5683574 Follow @expectgreatdata Slack join chat
contributors 401 Ruff

About GX Core

GX Core combines the collective wisdom of thousands of community members with a proven track record in data quality deployments worldwide, wrapped into a super-simple package for data teams.

Its powerful technical tools start with Expectations: expressive and extensible unit tests for your data. Expectations foster collaboration by giving teams a common language to express data

https://github.com/great-expectations/great_expectations

Deequ by AWS

Deequ - Unit Tests for Data

license Apache-2.0 issues 143 open Java CI with Maven passing Maven Central

Deequ is a library built on top of Apache Spark for defining "unit tests for data", which measure data quality in large datasets. We are happy to receive feedback and contributions.

Python users may also be interested in PyDeequ, a Python interface for Deequ. You can find PyDeequ on GitHub, readthedocs, and PyPI.

https://github.com/awslabs/deequ

Most Common Open Source Libraries for Basic Feature Validation

Great Expectations (Example)

Table shape

- expect_column_to_exist
- expect_table_columns_to_match_ordered_list
- expect_table_columns_to_match_set
- expect_table_row_count_to_be_between
- expect_table_row_count_to_equal
- expect_table_row_count_to_equal_other_table

Missing values, unique values, and types

- expect_column_values_to_be_unique
- expect_column_values_to_not_be_null
- expect_column_values_to_be_null
- expect_column_values_to_be_of_type
- expect_column_values_to_be_in_type_list

After feature validations, we can further perform two-sample testing

```
expect_column_values_to_be_between(
  column="room_temp",
  min_value=60,
  max_value=75,
  mostly=.95
)
```

→

"Values in this column should be between 60 and 75 at least 95% of the time."

"Warning: more than 5% of values fell outside the specified range of 60 to 75."

Figure 8-7. Some of the built-in feature validation functions by Great Expectations and an example of how to use them. Source: Adapted from content in the Great Expectations GitHub repository

Source: Chip Huyen, Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications. Chapter 8. O'Reilly Media, Inc. 2022

Four Major Concerns When Doing Feature Monitoring

1

A company might have hundreds of models in production, and each model uses hundreds, if not thousands, of features.

Too many metrics measuring can slow down your system.

2

While tracking features is useful for debugging purposes, it' s not very useful for detecting model performance degradation.

3

Feature extraction is often done in multiple steps (such as filling missing values and standardization), using multiple libraries (such as Pandas, Spark), on multiple services (such as BigQuery or Snowflake)

It might be impossible to detect if the critical changes caused by the input distribution or by multiple processing steps.

4

The schema that the features follow can change over time.

We need a way to do schema versioning and the mapping of features to specific version of a schema

Monitoring Raw Inputs

Track inputs, features, predictions, and accuracy to detect issues early

- **Raw inputs might not be easy to monitor**

Raw inputs can come from multiple sources.
Raw inputs are often managed by a data platform team to process and move processed data to the warehouse.
The ML engineers usually obtain processed data from the warehouse.

Source: Chip Huyen, Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications. Chapter 8. O'Reilly Media, Inc. 2022

Monitoring Toolbox

Measuring, tracking, and interpreting metrics for complex systems in a non-travial tasks

Logs

detailed runtime records and distributed tracing for debugging and root cause analysis.

Dashboards

visualizations of key metrics over time for engineers and business stakeholders.

Alerts

automated notifications for threshold breaches that must be meaningful to avoid alert fatigue.

Logs

detailed runtime records and distributed tracing for debugging and root cause analysis.

- **When we log an event, we want to make it as easy as possible for us to find it later**

  In microservice architecture, this is called “distributed tracing”
  We give each process a unique ID, so that, if the process goes wrong, we hope that we can find the error messages with that ID. Also, we want to log related meta data, e.g., called services and functions, the account associated with the process and so on.

- **The log management market was estimated to be doubled from 2021 to 2026.**
- **MLs have been used to analyze logs, anomaly detection.**
- **The log analysis can be used to know the probability of related services being affected, useful for handling system under cybersecurity attack**
- **Many companies process logs in batch processes and then query for specific events using SQL or process them via a Spark or Hadoop or Hive cluster. *Process logs periodically, discover problems periodically.***
- **To get events sooner, we can use real-time transport such as Kafka or Amazon Kinesis to transport events. To query in real-time, we can use SQL engine like KSQL**

```
sudo docker container logs training
```

[2026-02-18 16:47:00,436: INFO/Process-1] Connected to amqp://admin:**@10.8.0.1:5672//
[2026-02-18 16:47:00,818: INFO/Process-1] mingle: searching for neighbors
[2026-02-18 16:47:02,430: INFO/Process-1] mingle: all alone
[2026-02-18 16:47:03,579: INFO/Process-1] Task tasks.train_model_task[a60bf04a-54b7-44b7-b070-b02a33593caf] received
[2026-02-18 16:47:03,579: WARNING/Process-1] {"task_id": "a60bf04a-54b7-44b7-b070-b02a33593caf", "event": "[TASKS] Starting training task", "level": "info"}
[2026-02-18 16:47:03,579: WARNING/Process-1] {"model_id": "807358c5-a1ee-4b54-b3c5-c48afd29e8a9", "version": "5ad58c09-8bdc-42c2-8cb2-cdb4ff30a23b", "event": "[TASKS] Training model", "level": "info"}
[2026-02-18 16:47:03,579: WARNING/Process-1] {"event": "[PIPELINE] Starting pipeline...", "level": "info"}
[2026-02-18 16:47:03,731: WARNING/Process-1] {"filename": "5ad58c09-8bdc-42c2-8cb2-cdb4ff30a23b.onnx", "event": "[PIPELINE] Model already exists in GCS", "level": "warning"}
[2026-02-18 16:47:03,731: WARNING/Process-1] {"version": "5ad58c09-8bdc-42c2-8cb2-cdb4ff30a23b", "event": "[PIPELINE] Skipping training, version already completed", "level": "info"}
[2026-02-18 16:47:03,732: WARNING/Process-1] {"version": "5ad58c09-8bdc-42c2-8cb2-cdb4ff30a23b", "event": "[PIPELINE] Updating status to COMPLETED for existing model", "level": "info"}
[2026-02-18 16:47:03,936: WARNING/Process-1] {"message": "User model version status updated successfully", "event": "[GRPC] Status updated successfully", "level": "info"}
[2026-02-18 16:47:03,936: WARNING/Process-1] {"event": "[PIPELINE] Status updated to COMPLETED successfully", "level": "info"}
[2026-02-18 16:47:03,936: WARNING/Process-1] {"status": "COMPLETED", "event": "[TASKS] Task completed successfully", "level": "info"}
[2026-02-18 16:47:03,937: INFO/Process-1] Task tasks.train_model_task[a60bf04a-54b7-44b7-b070-b02a33593caf] succeeded in 0.3575671021826565s: {'status': 'COMPLETED', 'job_id': '5ad58c09-8bdc-42c2-8cb2-cdb4ff30a23b', 'model_name': 'microsoft/resnet-101', 'metrics': {'note': 0.0}, 'artifacts': {'onnx': 'gs://drills_storage/drills_model_weight/5ad58c09-8bdc-42c2-8cb2-cdb4ff30a23b.onnx', 'pytorch': 'gs://drills_storage/drills_model_pytorch/5ad58c09-8bdc-42c2-8cb2-cdb4ff30a23b.pt'}, 'error_message': None, 'completed_at': '2026-02-18T16:47:03.936459+00:00'}
{"model_id": "807358c5-a1ee-4b54-b3c5-c48afd29e8a9", "event": "[CONSUMER] Received message", "level": "info"}
{"model_id": "807358c5-a1ee-4b54-b3c5-c48afd29e8a9", "version": "f77dc5dd-c8fe-49d9-b6f8-1cdf263c3d36", "event": "[CONSUMER] Validated request", "level": "info"}
{"task_id": "6601a6cd-5647-4ef9-96cd-e26b7d183f34", "event": "[CONSUMER] Task sent to Celery", "level": "info"}
{"event": "[CONSUMER] Message acknowledged. Consumer ready for next message.", "level": "info"}
[2026-02-19 04:49:51,475: INFO/Process-1] Task tasks.train_model_task[6601a6cd-5647-4ef9-96cd-e26b7d183f34] received
[2026-02-19 04:49:51,475: WARNING/Process-1] {"task_id": "6601a6cd-5647-4ef9-96cd-e26b7d183f34", "event": "[TASKS] Starting training task", "level": "info"}
[2026-02-19 04:49:51,475: WARNING/Process-1] {"model_id": "807358c5-a1ee-4b54-b3c5-c48afd29e8a9", "version": "f77dc5dd-c8fe-49d9-b6f8-1cdf263c3d36", "event": "[TASKS] Training model", "level": "info"}
[2026-02-19 04:49:51,475: WARNING/Process-1] {"event": "[PIPELINE] Starting pipeline...", "level": "info"}

Dashboard

visualizations of key metrics over time for engineers and business stakeholders.

- **Dashboard is to make monitoring accessible to non-engineers, e.g., product managers, business developers**
- **Even though graphs can help a lot with understanding metrics, we still need experience and statistical knowledge**
- **Excessive metrics on a dashboard can be also counterproductive, a phenomenon known as *dashboard rot***
- **Pick the right metrics or abstract out the lower-level metrics to compute higher-level signals that make better sense for a specific task**

Alerts

Alerts the right person about something suspicious.

Three components in alerts

- **An alert policy**

  This describes the condition for an alert, e.g., you might want to create an alert when a metric breaches a threshold optionally over a certain duration. For examples, you may be notified when a model's accuracy in under 90% or the HTTP response latency is higher than a second for at least 10 minutes.

- **Notification channels**

  This describes who and how to be notified with the alerts, e.g., you might configure the alerts to be sent to an email address or to post to a Slack channel.

- **A description of the alert**

  This helps the alerted person understand what's going on. The description should be as detailed as possible. Ex.

```
## Recommender model accuracy below 90%
${timestamp}: This alert originated from the service ${service-name}
```

Observability

Finer grain of metrics

Telemetry: remote logs and metrics collected from remote components such as clod services or applications run on customer devices

1 **Since the mid-2010s, the industry started to embrace the term “observability” instead of “monitoring”**

Monitoring assumes no assumptions about the relationship between system’s internal states and its outputs

2 **Software systems grow much more complex and several components may be on cloud or with third-party.**

The team does not have control inside, hence, they can only rely on external outputs to figure out what’ is going on internally.

3 **Observability term is used to address above challenge**

Bring better visibility into understanding the complex behavior of software system using outputs collected from the system at run time.

Differences between Monitoring and Observability

Monitoring centers around metrics, and metrics are usually aggregated

Observability allows finer grain metrics so that you will know not only what the time period the model performance degrades but also what types of inputs/ what subgroup of users / etc.

Ex. Show me all users the model generates the wrong predictions over the last hour grouped by zip code...

Ex. Show me the outliers requests in the last to minutes

Ex. Show me all intermediate outputs of this input through the system